

Đây là notebook đi kèm cho cuốn sách [Deep Learning with Python, Third Edition](#). Để dễ đọc, notebook này chỉ chứa các khối mã có thể chạy được và các tiêu đề phần, đồng thời lược bỏ mọi thứ khác trong sách: các đoạn văn bản, hình minh họa và mã giả (pseudocode).

**Nếu bạn muốn hiểu rõ những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản sao cuốn sách của mình.**

Nội dung của cuốn sách có sẵn trực tuyến tại [deeplearningwithpython.io](#).

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

Hiện mã

- ✦ Giải mã những gì Mạng tích chập (ConvNets) học được
- ✦ Trực quan hóa các kích hoạt trung gian (intermediate activations)

```
from google.colab import files

# You can use this to load the file
# "convnet_from_scratch_with_augmentation.keras"
# you obtained in the last chapter.
files.upload()
```

```
import keras
model = keras.models.load_model(
    "convnet_from_scratch_with_augmentation.keras"
)
model.summary(line_length=80)
```

```
import keras
import numpy as np

img_path = keras.utils.get_file(
    fname="cat.jpg", origin="https://img-datasets.s3.amazonaws.com/cat.jpg"
)

def get_img_array(img_path, target_size):
    img = keras.utils.load_img(img_path, target_size=target_size)
    array = keras.utils.img_to_array(img)
    array = np.expand_dims(array, axis=0)
    return array

img_tensor = get_img_array(img_path, target_size=(180, 180))
```

```
import matplotlib.pyplot as plt

plt.axis("off")
plt.imshow(img_tensor[0].astype("uint8"))
plt.show()
```

```
from keras import layers

layer_outputs = []
layer_names = []
for layer in model.layers:
    if isinstance(layer, (layers.Conv2D, layers.MaxPooling2D)):
        layer_outputs.append(layer.output)
        layer_names.append(layer.name)
activation_model = keras.Model(inputs=model.input, outputs=layer_outputs)
```

```
activations = activation_model.predict(img_tensor)
```

```
first_layer_activation = activations[0]
print(first_layer_activation.shape)
```

```
import matplotlib.pyplot as plt
```

```
plt.matshow(first_layer_activation[0, :, :, 5], cmap="viridis")
```

```
images_per_row = 16
for layer_name, layer_activation in zip(layer_names, activations):
    n_features = layer_activation.shape[-1]
    size = layer_activation.shape[1]
    n_cols = n_features // images_per_row
    display_grid = np.zeros(
        ((size + 1) * n_cols - 1, images_per_row * (size + 1) - 1)
    )
    for col in range(n_cols):
        for row in range(images_per_row):
            channel_index = col * images_per_row + row
            channel_image = layer_activation[0, :, :, channel_index].copy()
            if channel_image.sum() != 0:
                channel_image -= channel_image.mean()
                channel_image /= channel_image.std()
                channel_image *= 64
                channel_image += 128
            channel_image = np.clip(channel_image, 0, 255).astype("uint8")
            display_grid[
                col * (size + 1) : (col + 1) * size + col,
                row * (size + 1) : (row + 1) * size + row,
            ] = channel_image
    scale = 1.0 / size
    plt.figure(
        figsize=(scale * display_grid.shape[1], scale * display_grid.shape[0])
    )
    plt.title(layer_name)
    plt.grid(False)
    plt.axis("off")
    plt.imshow(display_grid, aspect="auto", cmap="viridis")
```

## ✓ Trực quan hóa các bộ lọc (filters) của ConvNet

```
import keras_hub
```

```
model = keras_hub.models.Backbone.from_preset(
    "xception_41_imagenet",
)
preprocessor = keras_hub.layers.ImageConverter.from_preset(
    "xception_41_imagenet",
    image_size=(180, 180),
)
```

```
for layer in model.layers:
    if isinstance(layer, (keras.layers.Conv2D, keras.layers.SeparableConv2D)):
        print(layer.name)
```

```
layer_name = "block3_sepconv1"
layer = model.get_layer(name=layer_name)
feature_extractor = keras.Model(inputs=model.input, outputs=layer.output)
```

```
activation = feature_extractor(preprocessor(img_tensor))
```

```
from keras import ops
```

```
def compute_loss(image, filter_index):
    activation = feature_extractor(image)
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]
    return ops.mean(filter_activation)
```

## ✓ Gradient ascent trong TensorFlow

```
%%backend tensorflow
import tensorflow as tf

@tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        tape.watch(image)
        loss = compute_loss(image, filter_index)
        grads = tape.gradient(loss, image)
        grads = ops.normalize(grads)
        image += learning_rate * grads
    return image
```

## ▼ Gradient ascent trong PyTorch

```
%%backend torch
import torch

def gradient_ascent_step(image, filter_index, learning_rate):
    image = image.clone().detach().requires_grad_(True)
    loss = compute_loss(image, filter_index)
    loss.backward()
    grads = image.grad
    grads = ops.normalize(grads)
    image = image + learning_rate * grads
    return image
```

## ▼ Gradient ascent in JAX

```
%%backend jax
import jax

grad_fn = jax.grad(compute_loss)

@jax.jit
def gradient_ascent_step(image, filter_index, learning_rate):
    grads = grad_fn(image, filter_index)
    grads = ops.normalize(grads)
    image += learning_rate * grads
    return image
```

## ▼ Vòng lặp trực quan hóa bộ lọc

```
img_width = 200
img_height = 200

def generate_filter_pattern(filter_index):
    iterations = 30
    learning_rate = 10.0
    image = keras.random.uniform(
        minval=0.4, maxval=0.6, shape=(1, img_width, img_height, 3)
    )
    for i in range(iterations):
        image = gradient_ascent_step(image, filter_index, learning_rate)
    return image[0]
```

```
def deprocess_image(image):
    image -= ops.mean(image)
    image /= ops.std(image)
    image *= 64
    image += 128
    image = ops.clip(image, 0, 255)
    image = image[25:-25, 25:-25, :]
    image = ops.cast(image, dtype="uint8")
    return ops.convert_to_numpy(image)
```

```
plt.axis("off")
plt.imshow(deprocess_image(generate_filter_pattern(filter_index=2)))
```

```

all_images = []
for filter_index in range(64):
    print(f"Processing filter {filter_index}")
    image = deprocess_image(generate_filter_pattern(filter_index))
    all_images.append(image)

margin = 5
n = 8
box_width = img_width - 25 * 2
box_height = img_height - 25 * 2
full_width = n * box_width + (n - 1) * margin
full_height = n * box_height + (n - 1) * margin
stitched_filters = np.zeros((full_width, full_height, 3))

for i in range(n):
    for j in range(n):
        image = all_images[i * n + j]
        stitched_filters[
            (box_width + margin) * i : (box_width + margin) * i + box_width,
            (box_height + margin) * j : (box_height + margin) * j + box_height,
            :,
        ] = image

keras.utils.save_img(f"filters_for_layer_{layer_name}.png", stitched_filters)

```

## ✓ Trực quan hóa bản đồ nhiệt của các kích hoạt lớp (Class Activation Maps - CAM)

```

img_path = keras.utils.get_file(
    fname="elephant.jpg",
    origin="https://img-datasets.s3.amazonaws.com/elephant.jpg",
)
img = keras.utils.load_img(img_path)
img_array = np.expand_dims(img, axis=0)

```

```

model = keras_hub.models.ImageClassifier.from_preset(
    "xception_41_imagenet",
    activation="softmax",
)
preds = model.predict(img_array)
preds.shape

```

```
keras_hub.utils.decode_imagenet_predictions(preds)
```

```
np.argmax(preds[0])
```

```
img_array = model.preprocessor(img_array)
```

```

last_conv_layer_name = "block14_sepconv2_act"
last_conv_layer = model.backbone.get_layer(last_conv_layer_name)
last_conv_layer_model = keras.Model(model.inputs, last_conv_layer.output)

```

```

classifier_input = last_conv_layer.output
x = classifier_input
for layer_name in ["pooler", "predictions"]:
    x = model.get_layer(layer_name)(x)
classifier_model = keras.Model(classifier_input, x)

```

## ✓ Lấy gradient của lớp cao nhất: Phiên bản TensorFlow

```

%%backend tensorflow
import tensorflow as tf

def get_top_class_gradients(img_array):
    last_conv_layer_output = last_conv_layer_model(img_array)
    with tf.GradientTape() as tape:
        tape.watch(last_conv_layer_output)
        preds = classifier_model(last_conv_layer_output)
        top_pred_index = ops.argmax(preds[0])
        top_class_channel = preds[:, top_pred_index]

```

```

grads = tape.gradient(top_class_channel, last_conv_layer_output)
return grads, last_conv_layer_output

grads, last_conv_layer_output = get_top_class_gradients(img_array)
grads = ops.convert_to_numpy(grads)
last_conv_layer_output = ops.convert_to_numpy(last_conv_layer_output)

```

#### ▼ Lấy gradient của lớp cao nhất: Phiên bản PyTorch

```

%%backend torch
def get_top_class_gradients(img_array):
    last_conv_layer_output = last_conv_layer_model(img_array)
    last_conv_layer_output = (
        last_conv_layer_output.clone().detach().requires_grad_(True)
    )
    preds = classifier_model(last_conv_layer_output)
    top_pred_index = ops.argmax(preds[0])
    top_class_channel = preds[:, top_pred_index]
    top_class_channel.backward()
    grads = last_conv_layer_output.grad
    return grads, last_conv_layer_output

grads, last_conv_layer_output = get_top_class_gradients(img_array)
grads = ops.convert_to_numpy(grads)
last_conv_layer_output = ops.convert_to_numpy(last_conv_layer_output)

```

#### ▼ Lấy gradient của lớp cao nhất: Phiên bản JAX

```

%%backend jax
import jax

def loss_fn(last_conv_layer_output):
    preds = classifier_model(last_conv_layer_output)
    top_pred_index = ops.argmax(preds[0])
    top_class_channel = preds[:, top_pred_index]
    return top_class_channel[0]

grad_fn = jax.grad(loss_fn)

def get_top_class_gradients(img_array):
    last_conv_layer_output = last_conv_layer_model(img_array)
    grads = grad_fn(last_conv_layer_output)
    return grads, last_conv_layer_output

grads, last_conv_layer_output = get_top_class_gradients(img_array)
grads = ops.convert_to_numpy(grads)
last_conv_layer_output = ops.convert_to_numpy(last_conv_layer_output)

```

#### ▼ Hiển thị bản đồ nhiệt kích hoạt lớp

```

pooled_grads = np.mean(grads, axis=(0, 1, 2))
last_conv_layer_output = last_conv_layer_output[0].copy()
for i in range(pooled_grads.shape[-1]):
    last_conv_layer_output[:, :, i] *= pooled_grads[i]
heatmap = np.mean(last_conv_layer_output, axis=-1)

```

```

heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
plt.matshow(heatmap)

```

```

import matplotlib.cm as cm

img = keras.utils.load_img(img_path)
img = keras.utils.img_to_array(img)

heatmap = np.uint8(255 * heatmap)

jet = cm.get_cmap("jet")
jet_colors = jet(np.arange(256))[:, :3]

```

```
jet_heatmap = jet_colors[heatmap]

jet_heatmap = keras.utils.array_to_img(jet_heatmap)
jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape[0]))
jet_heatmap = keras.utils.img_to_array(jet_heatmap)

superimposed_img = jet_heatmap * 0.4 + img
superimposed_img = keras.utils.array_to_img(superimposed_img)

plt.imshow(superimposed_img)
```

Trực quan hóa không gian tiềm ẩn (latent space) của một ConvNet